



CHAPTER TWO

Number Systems and Codes

Outline

In this chapter:

- ❖ Numbering system
- ❖ Complements
- ❖ Fixed point representation
- ❖ Floating point representation
- ❖ Codes

Introduction To Number System

- ❖ Why Binary number?
- ❖ Inside the computer, there are integrated circuits with thousands of transistors.
- ❖ These transistors are made to **operate on a two-state**.
- ❖ By this design, all the input and output voltages are either **HIGH** or **LOW**.
- ❖ Low voltage represents binary 0 and high voltage represents binary 1.

❖	Voltage	Low	High
❖	Binary	0	1

Cont..

- ❖ In Computer the data is **represented in binary format** (1s and 0s).
- ❖ Even though we use characters, decimals, punctuation marks, symbols and graph, internally these things are represented in binary format.
- ❖ So we will go through the **number system**.
- ❖ There are different number systems namely decimal, binary, octal, hexadecimal, etc.
- ❖ Decimal number system is important because it is universally **used to represent quantities** outside a digital system.
 - Decimal system uses numbers from 0 to 9. This is to the base 10.
- ❖ Binary numbers are based on the concept of ON or OFF.
 - Binary number system has a base of 2.
 - Its two digits are denoted by 0 & 1 and are called **bits**.

Cont'd

- ❖ Octal Number system uses exactly eight symbols 0,1,2,3,4,5,6, and 7.
i.e., it has a base of 8.
 - Each octal digit has a unique 3 bit binary representation.
- ❖ Hexadecimal number system is to the base 16.
 - It spans from 0 to 9 and then A to F.
 - In hexadecimal system A,B,C,D,E and F represents 10,11,12,13,14 and 15, i.e.,
 - Hexadecimal numbers are more convenient for people to recognize and interpret than the long strings of binary numbers

Cont'd

❖ **Positional Numbering Systems**

- Radix (or Base): The general idea behind positional numbering systems is that a numeric value is represented through increasing powers of a **radix** (or base).

Cont..

System	Radix	Allowable Digits
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Binary	2	0, 1
Octal	8	0, 1, 2, 3, 4, 5, 6, 7
Hexadecimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Powers of 2
$2^{-2} = \frac{1}{4} = 0.25$
$2^{-1} = \frac{1}{2} = 0.5$
$2^0 = 1$
$2^1 = 2$
$2^2 = 4$
$2^3 = 8$
$2^4 = 16$
$2^5 = 32$
$2^6 = 64$
$2^7 = 128$
$2^8 = 256$
$2^9 = 512$
$2^{10} = 1,024$
$2^{15} = 32,768$
$2^{16} = 65,536$

Decimal	4-Bit Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Number System Conversions-

- ❖ It is very important to have a good knowledge of how to convert numbers from one base to another base.
- ❖ Here, we will learn how to convert any given number from any base to any other base.



The diagram is enclosed in a black rectangular border. It shows a conversion process from left to right. On the left, the text "(Given Number)" is in orange, followed by "base x" in a smaller orange font. A black arrow points to the right. On the right, the text "(?)" is in orange, followed by "base y" in a smaller orange font.

(Given Number) base x $\longrightarrow$ (?) base y

Cont..

A given number in base x can be converted to any other base y using the following steps-

Step-01:- Convert the number from base x to base 10 using expansion method

- A given number can be converted from any base to base 10 using Expansion Method.
- According to expansion method, if abc.de is any given number in base x, then its value in base 10 is given as-

$$(abc.de)_x = (ax^2 + bx + c + dx^{-1} + ex^{-2})_{10}$$

Cont..

To use expansion method for conversion,

- Assign position number to each digit of the given number.
- Digits to the left of decimal are numbered starting from 0.
- Digits to the right of decimal are numbered starting from -1.
- Write a term for each digit as $\text{digit} \times (\text{base of given number})^{\text{position number of digit}}$
- Perform the addition of all terms to obtain the number in base 10.
- This formula can be expanded for any number of digits.

Cont..

$$\begin{array}{cccccc} & 2 & 1 & 0 & -1 & -2 \\ & (& a & b & c & . & d & e &) & x \\ & \swarrow & \swarrow & \swarrow & \swarrow & \swarrow & & & \\ (& ax^2 & + & bx^1 & + & cx^0 & + & dx^{-1} & + & ex^{-2} &)_{10} \end{array}$$

Expansion Method

Cont..

Convert the following numbers to base 10-

1. $(10010)_2$

2. $(254)_8$

3. $(AC)_{16}$

4. $(10010.101)_2$

5. $(254.7014)_8$

6. $(AC.FBA5)_{16}$

7. $(0.1402)_8$

8. $(0.ABDF)_{16}$

Cont..

1. $(10010)_2$

$(10010)_2 \rightarrow (?)_{10}$

Using expansion method, we have-

$$(10010)_2 = (1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0)_{10}$$

$$= (16 + 0 + 0 + 2 + 0)_{10}$$

$$= (18)_{10}$$

cont..

2, (254)₈

$$(254)_8 \rightarrow (?)_{10}$$

Using expansion method, we have-

$$(254)_8 = (2 \times 8^2 + 5 \times 8^1 + 4 \times 8^0)_{10}$$

$$= (128 + 40 + 4)_{10}$$

$$= (172)_{10}$$

3. (AC)₁₆

$$(AC)_{16} \rightarrow (?)_{10}$$

Using expansion method, we have-

$$\begin{aligned}(AC)_{16} &= (A \times 16^1 + C \times 16^0)_{10} \\&= (10 \times 16 + 12 \times 1)_{10} \\&= (160 + 12)_{10} \\&= (172)_{10}\end{aligned}$$

4. (10010.101)₂

$$(10010.101)_2 \rightarrow (?)_{10}$$

Using expansion method, we have-

$$(10010.101)_2$$

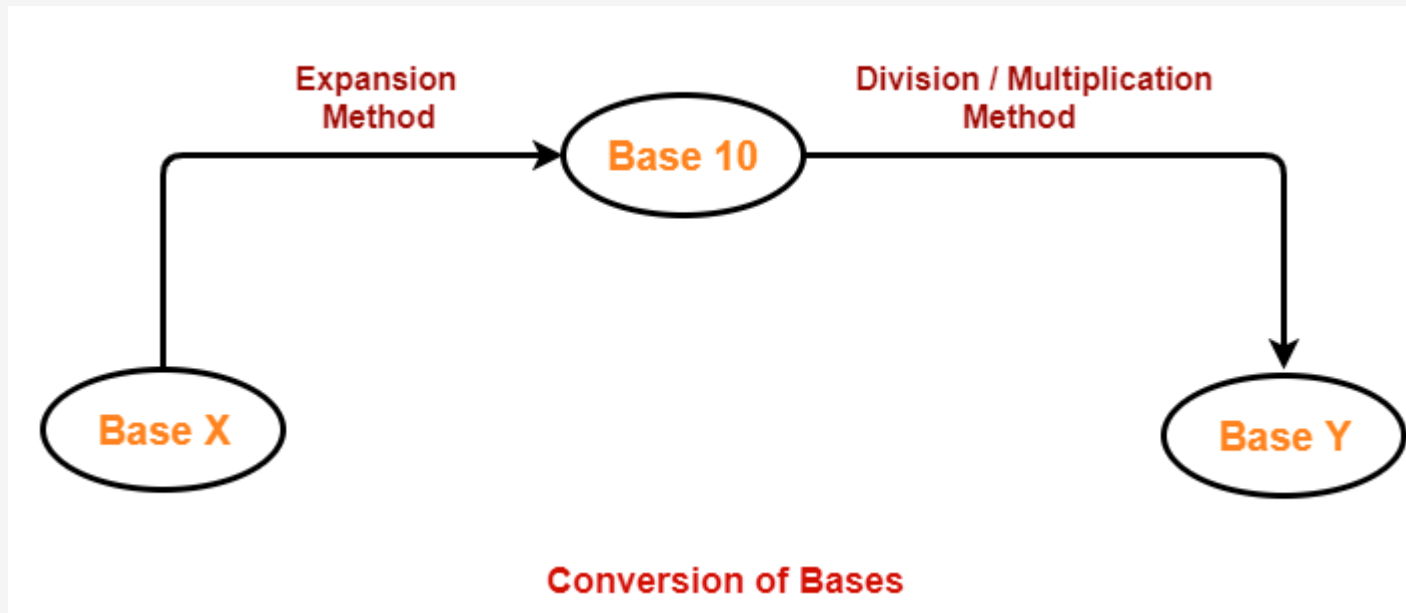
$$= (1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3})_{10}$$

$$= (16 + 0 + 0 + 2 + 0 + 0.5 + 0.125)_{10}$$

$$= (18.625)_{10}$$

Step-02:

- ❖ Convert the number from base 10 to base y using division & multiplication method.

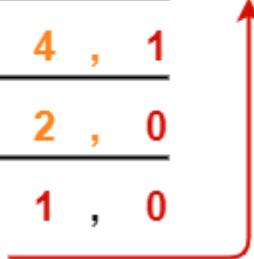


Decimal to Binary Conversion-

- ❖ A given number can be converted from base 10 to any other base using division method and multiplication method.

Exercise $(18)_{10} \rightarrow (?)_2$

2	18
2	9 , 0
2	4 , 1
2	2 , 0
	1 , 0



Cont..

Exercise 2 $(18.625)_{10} \rightarrow (?)_2$

Solution

for fractional part use the multiplicity Ruel

integer part

$$0.625 * 2 = 1.25$$

1

$$0.25 * 2 =$$

0

$$0.5 * 2 = 1$$

1



$$(18.625)_{10} = 10010.101$$

Decimal to Octal Conversion | Base 10 to Base 8

Steps To Convert From Base 10 to Base 8-

- Divide the given number (in base 10) with 8 until the result finally left is less than 8.
- Traverse the remainders from bottom to top to get the required number in base 8.

Case-02: For Numbers Carrying A Fractional Part-

- To convert such numbers from base 10 to another base, real part and fractional part are treated separately.

For Real Part-

- ✓ The steps involved in converting the real part from base 10 to another base are same as above.

For Fractional Part-

- ✓ Multiplication Method is used to convert fractional part from base 10 to another base.
- ✓ The multiplication is performed with the required base.

Cont..

E.g. $(1032.6875)_{10} \rightarrow (?)_8$

For Real Part:- The real part is $(1032)_{10}$

We convert the real part from base 10 to base 8 using division method same as above.

So, $(1032)_{10} = (2010)_8$

For Fractional Part-

The fractional part is $(0.6875)_{10}$

We convert the fractional part from base 10 to base 8 using multiplication method.

-

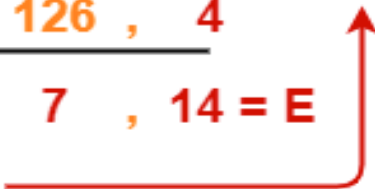
	Real part	Fractional Part
0.6875×8	5	0.5
0.5×8	4	0.0

Decimal to Hexadecimal | Base 10 to base 16

Example $(2020)_{10} \rightarrow (?)_{16}$

❖ Using division method, we have-

16	2020
16	126 , 4
	7 , 14 = E



❖ From here, $(2020)_{10} = (7E4)_{16}$

Others

Binary to hexadecimal or octal conversion examples

$$11010110_2 = \overbrace{011} \overbrace{010} \overbrace{110}_2 = 326_8$$

$$= \overbrace{1101} \overbrace{0110}_2 = D6_{16}$$

$$11010010.10110_2 = \overbrace{0110} \overbrace{10010} \overbrace{.1011} \overbrace{00}_2 = 322.54_8$$

$$= \overbrace{1101} \overbrace{0010} \overbrace{.1011}_2 = D2.B_{16}$$

complement

- ❖ Generally, there are two types of complement of Binary number: 1's complement and 2's complement.
- ❖ To get 1's complement of a binary number, simply invert the given number. For example, 1's complement of binary number 110010 is 001101. To get 2's complement of binary number is 1's complement of given number plus 1 to the least significant bit (LSB). For example 2's complement of binary number 10010 is $(01101) + 1 = 01110$.

Complements..

- ❖ Generally, there are two types of complement of Binary number: 1's complement and 2's complement.
- ❖ **1's complement** :- it is a binary number is another binary number obtained by toggling all bits in it, i.e., transforming the 0 bit to 1 and the 1 bit to 0.

Examples::-Let numbers be stored using 4 bits

- ✓ 1's complement of 7 (0111) is 8 (1000)
- ✓ 1's complement of 12 (1100) is 3 (0011)

❖ **2's complement:-** 2's complement of a binary number is 1 added to the 1's complement of the binary number.

Examples:-Let numbers be stored using 4 bits

- ✓ 2's complement of 7 (0111) is 9 (1001)
- ✓ 2's complement of 12 (1100) is 4 (0100)
- ✓ To get 2's complement of binary number is 1's complement of given number plus 1 to the least significant bit (LSB).
- ✓ For example 2's complement of binary number 10010 is $(01101) + 1 = 01110$

Example: 1101 and -1001

1. First, find the 2's complement of the negative number 1001. 1001.
 - ✓ The 1's complement of the number 1001 is 0110, and add 1 to the LSB of the result 0110.
 - ✓ So the 2's complement of number 1001 is $0110+1=0111$
2. Add both the numbers, i.e., 1101 and 0111;
 $1101+0111=1\ 0100$
3. By adding both numbers, we get the end-around carry 1. We discard the **end-around carry**. So, the addition of both numbers is 0100

Fixed-Point Representation –

- ❖ This representation has fixed number of bits for integer part and for fractional part.
- ❖ For example, if given fixed-point representation is IIII. FFFF, then you can store minimum value is 0000.0001 and maximum value is 9999.9999.
- ❖ There are three parts of a fixed-point number representation: the sign field, integer field, and fractional field.



Cont.

- ❖ **Example** Assume number is using 32-bit format which reserve 1 bit for the sign, 15 bits for the integer part and 16 bits for the fractional part.
- ❖ Then, -43.625 is represented as following:

1	000000000101011	1010000000000000
Sign bit	Integer part	Fractional part

Floating new

❖floating point theory is very complex. The following example is used to offer a lead into the complex theory behind floating point representation. There are additional steps that are not relevant at this level of study.

❖The number 18.5 can be converted to binary to give us:

The number 18.5 can be converted to binary to give us=**10010.1**

-
- ❖ To store this as a floating-point number, we need to move the decimal point so that it appears after the most significant bit (MSB). The MSB is the bit on the left of the number. In this case we move the decimal point five places, so our number becomes: **0.100101 x 2⁵**
 - ❖ But we also need to store the value 5 (seen as 2⁵) as a binary number, so it would become: **0.100101 x 2⁰¹⁰¹**



- ❖ The 2⁰¹⁰¹ (2 to the power of 5) tells us that we need to move the decimal point five places to the right to recreate the original value.

Cont..

The system will not store the actual decimal place, it is shown to aid understanding. In this case the number is stored as follows

Sign bit	Exponent	mantissa
0	0101	0100101

~~Representation of Numbers, Operand of Code and Address~~

- ❑ Other binary codes for decimal numbers and alphanumeric characters are sometimes used.
- ❑ **Gray code::** This is an unweighted code. It means that there are no specific weights assigned to the bit position.
 - This code is not suitable for arithmetic operations, but it is very useful for input-output devices, Analog to digital conversion etc., and the machines which use Shaft encoders as sensors.
 - The following table gives the comparison of decimal, binary and gray codes.

Decimal	Binary Code	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101

Cont'd

- ❑ Note: Left most bit is the first binary digit as the first gray code.
- ❑ Then move from **left to right and add each adjacent pair of binary digit to get the next** gray code digit, neglect carries if any.

Ex: 1010 = ?

(1)	(2)	(3)	(4)
1	0	1	0

Add (1) + (2) $\rightarrow 1 + 0 = 1$ and Add (2) + (3) $\rightarrow 0 + 1 = 1$ and Add (3) + (4) $\rightarrow 1 + 0 = 1$ then result is 1111 $\rightarrow$ gray code

Cont'd

❑ BCD(Binary –Coded decimal)

- it is based on the idea of **converting each** decimal digits into its equivalent binary number.
- It means each decimal is represented by binary code of 4 bits.
- The devices such as Electronic calculators, Digital voltmeters, frequency counters, electronic counters, digital clocks etc, work with BCD numbers.
- Modern computers do not use BCD numbers as they have to process names and other non numeric data.

Decimal	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	0001 0000
11	0001 0001
12	0001 0010
.....	
77	0111 0111

Cont'd

❑ **Hexadecimal Versus BCD**

- The Hexadecimal system utilizes the full capacity of four binary bits, whereas BCD codes do not utilize the same.
- The BCD codes do not utilize the binary codes from 1010 to 1111.
- In the hexadecimal system an 8 bit word can represent up to FF, that is 11111111 (255 decimal) whereas in BCD only up to 10011001 (99 decimal).
- Hence the hexadecimal is a compact form of representation, and it occupies less memory space, thereby reducing the hardware cost.
- The arithmetic operations are also simpler in hexadecimal system.

Cont'd

- ❑ **Alpha numeric Code:** ASCII (American standard code for information interchange)
 - It is the standardized alphanumeric code and most widely used by several computer manufacturers as their computer's internal code.
 - It is a 7 bit alphanumeric code which has $2^7 = 128$ different characters to encode 85 characters. (52 lowercase and uppercase alphabets, 10 numerals and 23 punctuation and others symbols as marked on keyboards printers, video display etc.,)
 - ASCII code can be classified into 2 types as specified below.
 - ASCII -7 : 7 bit code, which represents $2^7 = 128$ different characters. In 7 bits, the first 3 bit represent zone bit and next 4 bit represent digit.
 - ASCII-8 : 8 bit code, which represent $2^8 = 256$ different characters. In 8 bits, the first 4 bit represent zone bit and next 4 bit represent digit.

Cont'd

❖ ASCII 7 code for characters

LSB	MSB $b_6b_5b_4$					
$b_3b_2b_1b_0$	010	011	100	101	110	111
0000	SPACE	0	@	P	-	P
0001	!	1	A	Q	A	Q
0010	“	2	B	R	B	R
0011	#	3	C	S	C	S
0100	\$	4	D	T	D	T
0101	%	5	E	U	E	U
0110	&	6	F	V	F	V
0111	‘	7	G	W	G	W
1000	(	8	H	X	H	X
1001	)	9	I	Y	I	Y
1010	*	:	J	Z	J	Z
1011	+	;	K	[	K	{
1100	‘	<	L	\	L	
1101	-	=	M	]	M	}
1110	.	>	N	^	N	~
1111	/	?	O	_	O	Del

- ❖ For instance, the letter ‘A’ has the bits $b_6b_5b_4$ of 100 and the bits $b_3b_2b_1b_0$ of 0001. Therefore, A is represented in ASCII code as 100 0001 $\rightarrow$ 41 (hexadecimal)

Thank You !!!